

18. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

volatile sig_atomic_t signal_received = 0;

/* Signal handler */
void handleSignal(int sig) {
    signal_received = sig;
    if (sig == SIGINT) {
        printf("\nReceived SIGINT (Ctrl+C)\n");
    }
    else if (sig == SIGTERM) {
        printf("Received SIGTERM\n");
    }
    else if (sig == SIGUSR1) {
        printf("Received SIGUSR1\n");
    }
}

/* Display signal mask */
void displaySignalMask() {
    sigset_t mask;
    sigprocmask(SIG_SETMASK, NULL, &mask);
    printf("\nCurrent Signal Mask:\n");
    printf("SIGINT : %s\n",
        sigismember(&mask, SIGINT) ? "Blocked" : "Not Blocked");
    printf("SIGTERM : %s\n",
        sigismember(&mask, SIGTERM) ? "Blocked" : "Not Blocked");
    printf("SIGUSR1 : %s\n",
        sigismember(&mask, SIGUSR1) ? "Blocked" : "Not Blocked");
}

int main() {
    pid_t pid;
    int status;

    printf("Signal Delivery and Process Group Demo\n");
    printf("-----\n");

    /* Install signal handlers */
```

```
/* Install signal handlers */
signal(SIGINT, handleSignal);
signal(SIGTERM, handleSignal);
signal(SIGUSR1, handleSignal);

printf("Parent PID      : %d\n", getpid());
printf("Parent Group ID : %d\n", getpgrp());

displaySignalMask();

/* Create child */
pid = fork();

if (pid < 0) {
    perror("fork");
    return 1;
}

if (pid == 0) {
    /* Child process */
    printf("\nChild Process\n");
    printf("-----\n");

    printf("Child PID      : %d\n", getpid());
    printf("Child Parent   : %d\n", getppid());
    printf("Child Group ID : %d\n", getpgrp());

    printf("\nWaiting for signals...\n");

    for (int i = 0; i < 10; i++) {
        sleep(1);
        if (signal_received != 0) {
            break;
        }
    }

    printf("Child process exiting.\n");
    exit(0);
}

/* Parent process */
sleep(1);
printf("\nParent sending SIGUSR1 to child...\n");
```

```

    }

    printf("Child process exiting.\n");
    exit(0);
}

/* Parent process */

sleep(1);

printf("\nParent sending SIGUSR1 to child...\n");

if (kill(pid, SIGUSR1) == -1) {
    perror("kill");
}

sleep(1);

printf("\nParent sending SIGTERM to child...\n");

if (kill(pid, SIGTERM) == -1) {
    perror("kill");
}

/* Wait for child */
waitpid(pid, &status, 0);

printf("\nChild process completed.\n");

if (WIFEXITED(status)) {
    printf("Child exit status: %d\n",
           WEXITSTATUS(status));
}

printf("\nSignal behavior test completed.\n");

return 0;
}

```

Result:

```

saisharan134@APL-5CD1366K5L:~/SKILL$ nano Skill18a.c
saisharan134@APL-5CD1366K5L:~/SKILL$ gcc Skill18a.c -o s18a
saisharan134@APL-5CD1366K5L:~/SKILL$ ./s18a
Signal Delivery and Process Group Demo
-----
Parent PID      : 3182
Parent Group ID : 3182

Current Signal Mask:
SIGINT  : Not Blocked
SIGTERM : Not Blocked
SIGUSR1 : Not Blocked

Child Process
-----
Child PID      : 3183
Child Parent   : 3182
Child Group ID : 3182

Waiting for signals...

Parent sending SIGUSR1 to child...
Received SIGUSR1
Child process exiting.

Parent sending SIGTERM to child...

Child process completed.
Child exit status: 0

Signal behavior test completed.
saisharan134@APL-5CD1366K5L:~/SKILL$

```

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 5
#define MAX_COMMAND 100

typedef enum {
    RUNNING,
    STOPPED,
    COMPLETED
} JobState;

typedef struct {
    int job_id;
    pid_t pid;
    char command[MAX_COMMAND];
    JobState state;
} Job;

Job jobTable[MAX_JOBS];
int jobCount = 0;

/* Add job to job table */
void addJob(pid_t pid, const char *command) {
    if (jobCount >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }

    jobTable[jobCount].job_id = jobCount + 1;
    jobTable[jobCount].pid = pid;
    strcpy(jobTable[jobCount].command, command);
    jobTable[jobCount].state = RUNNING;
    jobCount++;
}

/* Display job table */
void displayJobs() {
    printf("\nJob Table\n");
    printf("-----\n");
    printf("ID\tPID\tSTATE\t\tCOMMAND\n");
    printf("-----\n");

    for (int i = 0; i < jobCount; i++) {
        char *state;

        if (jobTable[i].state == RUNNING)
            state = "RUNNING";
        else if (jobTable[i].state == STOPPED)
            state = "STOPPED";
        else
            state = "COMPLETED";

        printf("[%d]\t%d\t%s\t\t%s\n",
            jobTable[i].job_id,
            jobTable[i].pid,
            state,
            jobTable[i].command);
    }

    printf("-----\n");
}

/* Find job by ID */
int findJob(int jobID) {
    for (int i = 0; i < jobCount; i++) {
        if (jobTable[i].job_id == jobID)
            return i;
    }

    return -1;
}

/* Update job status without blocking */
void updateJobStatus() {
    int status;
    for (int i = 0; i < jobCount; i++) {
        if (jobTable[i].state == RUNNING ||
            jobTable[i].state == STOPPED) {
            pid_t result = waitpid(
                jobTable[i].pid,
                &status,
                WNOHANG | WUNTRACED | WCONTINUED);
        }
    }
}
```

```

        &status,
        WNOHANG | WUNTRACED | WCONTINUED
    );
    if (result == jobTable[i].pid) {
        if (WIFSTOPPED(status)) {
            jobTable[i].state = STOPPED;
        } else if (WIFCONTINUED(status)) {
            jobTable[i].state = RUNNING;
        } else if (WIFEXITED(status) ||
                    WIFSIGNALED(status)) {
            jobTable[i].state = COMPLETED;
        }
    }
}
}
}
}

/* Stop a running job */
void stopJob(int jobID) {
    int index = findJob(jobID);
    if (index == -1) {
        printf("Job [%d] not found.\n", jobID);
        return;
    }
    if (jobTable[index].state != RUNNING) {
        printf("Job [%d] is not running.\n", jobID);
        return;
    }
    if (kill(jobTable[index].pid, SIGSTOP) == -1) {
        perror("SIGSTOP");
        return;
    }
    jobTable[index].state = STOPPED;
    printf("Job [%d] stopped successfully.\n", jobID);
}

```

```

    printf("Job [%d] stopped successfully.\n", jobID);
}

/* Resume stopped job */
void resumeJob(int jobID) {
    int index = findJob(jobID);
    if (index == -1) {
        printf("Job [%d] not found.\n", jobID);
        return;
    }
    if (jobTable[index].state != STOPPED) {
        printf("Job [%d] is not stopped.\n", jobID);
        return;
    }
    printf("Sending SIGCONT to Job [%d]...\n",
           jobID);
    if (kill(jobTable[index].pid, SIGCONT) == -1) {
        perror("SIGCONT");
        return;
    }
    jobTable[index].state = RUNNING;
    printf("Job [%d] resumed successfully.\n",
           jobID);
}

/* Create background job */
void createJob() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return;
    }
    if (pid == 0) {
        printf("Child process started. PID = %d\n",
               getpid());
    }
}

```

```

    /*
     * Run for 10 seconds.
     * The parent can stop and resume this job.
     */
    for (int i = 1; i <= 10; i++) {
        printf("Child working... %d/10\n", i);
        sleep(1);
    }

    printf("Child process completed.\n");
    exit(0);
}

addJob(pid, "background-task");

printf("Job [%d] created with PID %d\n",
       jobTable[jobCount - 1].job_id,
       pid);
}

int main() {
    printf("Stopped Job Recovery System\n");
    printf("-----\n");

    /* Create background job */
    createJob();

    sleep(2);

    /* Display initial state */
    updateJobStatus();
    displayJobs();

    /* Stop the job */
    printf("\nStopping Job [1]...\n");

    stopJob(1);

    updateJobStatus();
    displayJobs();

    /* Wait while job is stopped */
    printf("\nJob remains stopped for 2 seconds...\n");
    sleep(2);

    /* Resume the job */
    printf("\nResuming Job [1]...\n");
}

```

```

/* Wait while job is stopped */
printf("\nJob remains stopped for 2 seconds...\n");
sleep(2);

/* Resume the job */
printf("\nResuming Job [1]...\n");

resumeJob(1);

updateJobStatus();
displayJobs();

/* Wait for completion */
printf("\nWaiting for resumed job to finish...\n");

waitpid(jobTable[0].pid, NULL, 0);

jobTable[0].state = COMPLETED;

/* Final state */
printf("\nFinal Job State:\n");

displayJobs();

printf("\nBackground recovery completed successfully.\n");

return 0;
}

```

Result:

```
ketana_sindhu@KetaZone:~$ nano Skill18b.c
ketana_sindhu@KetaZone:~$ gcc Skill18b.c -o s18b
ketana_sindhu@KetaZone:~$ ./s18b
Stopped Job Recovery System

Job [1] created with PID 3296
Child process started. PID = 3296
Child working... 1/10
Child working... 2/10

Child working... 3/10
Job Table
-----
ID      PID      STATE      COMMAND
-----
[1]     3296     RUNNING    background-task

Stopping Job [1]...
Job [1] stopped successfully.

Job Table
-----
ID      PID      STATE      COMMAND
-----
[1]     3296     STOPPED    background-task

Job remains stopped for 2 seconds...

Resuming Job [1]...
Sending SIGCONT to Job [1]...
Job [1] resumed successfully.

Job Table
-----
ID      PID      STATE      COMMAND
-----
[1]     3296     RUNNING    background-task

Waiting for resumed job to finish...
Child working... 4/10
Child working... 5/10
Child working... 6/10
Child working... 7/10
Child working... 8/10
Child working... 9/10
Child working... 10/10
Child process completed.

Final Job State:

Job Table
-----
ID      PID      STATE      COMMAND
-----
[1]     3296     COMPLETED background-task

Background recovery completed successfully.
ketana_sindhu@KetaZone:~$ |
```